# EXPERIMENT 15

## Career Pathways and Future of Low-Code Development

### Aim

To study the career pathways in low-code development, to identify the skills required for different roles, and to understand the future growth and evolution of low-code platforms.

### Introduction to Low-Code Development

Low-code development is an approach to building software applications mainly through visual tools and configuration instead of writing large amounts of traditional program code. Applications are assembled using visual development environments, predefined components and templates. Typical building blocks include forms for capturing data, reports and dashboards for viewing it, and workflows that define how information moves through a business process.

Automation is a central feature of low-code platforms. Rules, triggers and scheduled actions can perform repetitive tasks without manual effort. Because much of the routine programming is handled by the platform, development effort is reduced and applications can be delivered faster. Low-code platforms are also accessible to a wider group of people: professional developers use them to speed up their work, and business users with limited programming experience can take part in building simple applications.

However, low-code does not remove the need for technical understanding. Concepts such as data models, logic, integration and security remain important, and developers who understand programming and technical principles are better able to design applications, extend them where the platform falls short and resolve problems when they occur.

### Career Pathways in Low-Code Development

A range of professional roles is associated with low-code development. They differ in the depth of technical knowledge and the scope of responsibility. The major roles are described below.

| Role | Focus | Typical Responsibilities |
|---|---|---|
| Low-Code Application Developer | Building applications on low-code platforms | Creating forms, reports and workflows; configuring application logic and automation. |
| Business Application Developer | Applications for specific business processes | Converting business requirements into working applications. |
| Automation Developer | Process automation | Designing automated workflows; reducing repetitive manual processes using triggers, workflows and automation tools. |
| Integration Developer | Connecting systems | Working with APIs and external services; connecting applications and systems; handling data exchange between platforms. |
| Low-Code Consultant | Advising on low-code solutions | Understanding business requirements; recommending suitable low-code solutions; helping organisations adopt low-code technologies. |
| Solution Developer / Platform Specialist | Advanced applications | Building more advanced applications; handling customisation and platform capabilities; working with integrations and complex requirements. |

| Role | Focus | Typical Responsibilities |
|---|---|---|
| Solution Architect | Solution design | Designing larger application solutions; planning architecture, integration and scalability; connecting business requirements with technical implementation. |
| Technical Lead / Enterprise Application Architect | Enterprise-level guidance | Guiding enterprise-level application development; handling technical decisions, governance, scalability and integration. |

Figure 1 shows how these roles fit into a progression. The path begins with a Foundation stage (basic programming, database concepts, UI/UX fundamentals, and workflow and automation), moves to the Low-Code Developer stage (application development, forms and reports, and business workflows), and then to Specialized Roles (Automation Developer, Integration Developer and Business Application Developer). Advanced Roles (Low-Code Consultant, Solution Developer and Platform Specialist) follow, and the path leads to Leadership roles (Solution Architect, Technical Lead and Enterprise Application Architect). The strip at the bottom of the figure lists the core skills that support all of these stages.

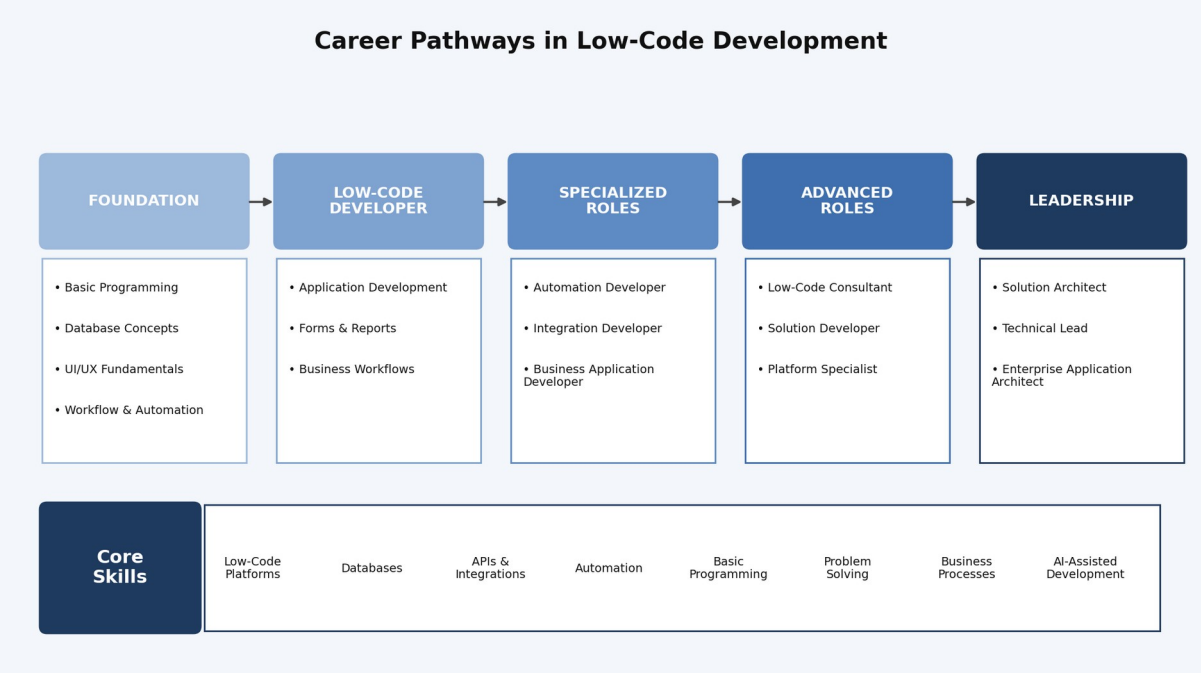



***Figure 1: Career progression and professional roles in low-code development.***

## Skills Required for Low-Code Careers

Low-code careers require a combination of platform knowledge and wider technical and professional skills.

| Skill | Relevance |
|---|---|
| Low-Code Platform Knowledge | Understanding the tools, components and limits of a platform in order to build applications efficiently. |
| Database Concepts | Designing tables, fields and relationships so that application data is organised correctly. |
| Workflow and Automation | Modelling business processes and automating repetitive tasks using rules and triggers. |

| Skill | Relevance |
|---|---|
| APIs and Integrations | Connecting applications with other systems and exchanging data between them. |
| Basic Programming | Writing scripts and custom logic where standard components are not sufficient. |
| UI/UX Fundamentals | Designing screens that are clear, consistent and easy for users to work with. |
| Problem Solving | Analysing requirements and difficulties and choosing suitable solutions. |
| Business Process Understanding | Understanding how an organisation works so that applications support real needs. |
| Data Handling | Managing, validating, processing and reporting on data. |
| AI-Assisted Development | Using AI tools to assist in building, testing and improving applications. |
| Communication and Requirement Analysis | Gathering requirements from users and explaining solutions to technical and non-technical people. |

Low-code development does not mean that technical knowledge becomes unnecessary. Developers increasingly benefit from combining platform knowledge with programming, databases, APIs, automation and business understanding.

## Career Progression

A realistic progression in low-code development is shown below. Responsibilities and technical depth generally increase at each stage.

| Stage | Description |
|---|---|
| Foundation | Basic programming, database concepts, UI/UX fundamentals and an understanding of workflow and automation. |
| Low-Code Developer | Building applications with forms, reports and workflows on a low-code platform. |
| Specialized Developer | Focusing on an area such as automation, integration or business applications. |
| Advanced Developer / Consultant | Handling complex requirements, customisation and advising organisations on solutions. |
| Solution Architect | Designing larger solutions, including architecture, integration and scalability. |
| Technical Lead / Enterprise Architect | Guiding enterprise-level development, technical decisions and governance. |

Beginners can start with platform fundamentals, forms, reports, workflows and basic databases. With experience, they can progress towards APIs, integrations, custom functions, advanced automation, architecture and enterprise solutions. As a person progresses, the work moves from building individual application components to designing complete solutions and guiding technical decisions.

24A0182

## Future of Low-Code Development

Low-code development is expected to become increasingly integrated with artificial intelligence, automation, cloud services and enterprise systems. The major trends are as follows:

- **AI-assisted application development:** AI tools that help in creating and improving applications.
- **Intelligent workflow automation:** automation that is supported by intelligent features.
- **Natural-language application building:** describing an application in everyday language to help build it.
- **API and cloud integration:** closer connection with cloud services and other systems through APIs.
- **Citizen development:** wider participation by business users in building applications.
- **Enterprise adoption:** use of low-code in larger organisations and for larger applications.
- **Low-code and pro-code collaboration:** combined use of visual development and custom code.
- **AI-powered testing and optimization:** AI support in testing applications and improving their performance.
- **Faster application delivery:** shorter time from requirement to working application.
- **Evolving developer roles:** changes in the work that developers do.

Figure 2 presents these trends, beginning with current low-code and moving through AI-assisted development, intelligent automation, enterprise low-code and hybrid low-code with pro-code towards future application development. It also shows the future developer as a combination of low-code platforms, programming fundamentals, APIs and integration, databases, AI tools and business understanding. The statement in the figure summarises the outlook: low-code will not eliminate developers but will change how they build, automate, integrate and solve business problems.

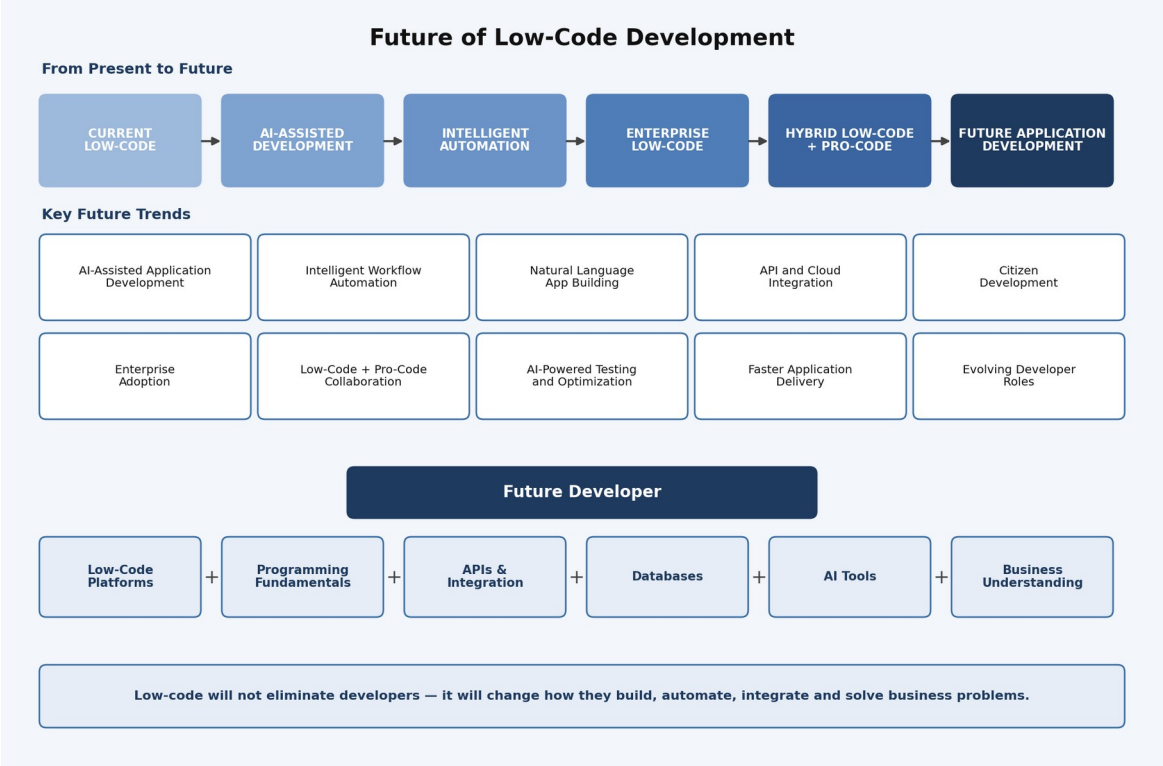



***Figure 2: Major trends shaping the future of low-code development.***

## Low-Code + AI + Pro-Code

Low-code, AI and pro-code complement one another rather than competing. Each contributes different strengths to application development.

| Low-Code | AI | Pro-Code |
|---|---|---|
| Faster application development | Intelligent automation | Custom business logic |
| Visual development | Natural-language interaction | Advanced processing |
| Pre-built components | Application assistance | APIs and integrations |
| Workflow automation | Data analysis and insights | Greater control |
| Accessibility | Testing and optimization | Specialized functionality |

Low-code provides the foundation of the application, AI adds intelligence and efficiency, and pro-code enables customisation. Figure 3 shows the three areas overlapping, together aiming at smarter applications for a digital future, with the expected results of faster development, higher productivity, better customisation, smarter decision making, improved collaboration and long-term scalability. Future developers may therefore need to combine all three rather than rely on only one approach.

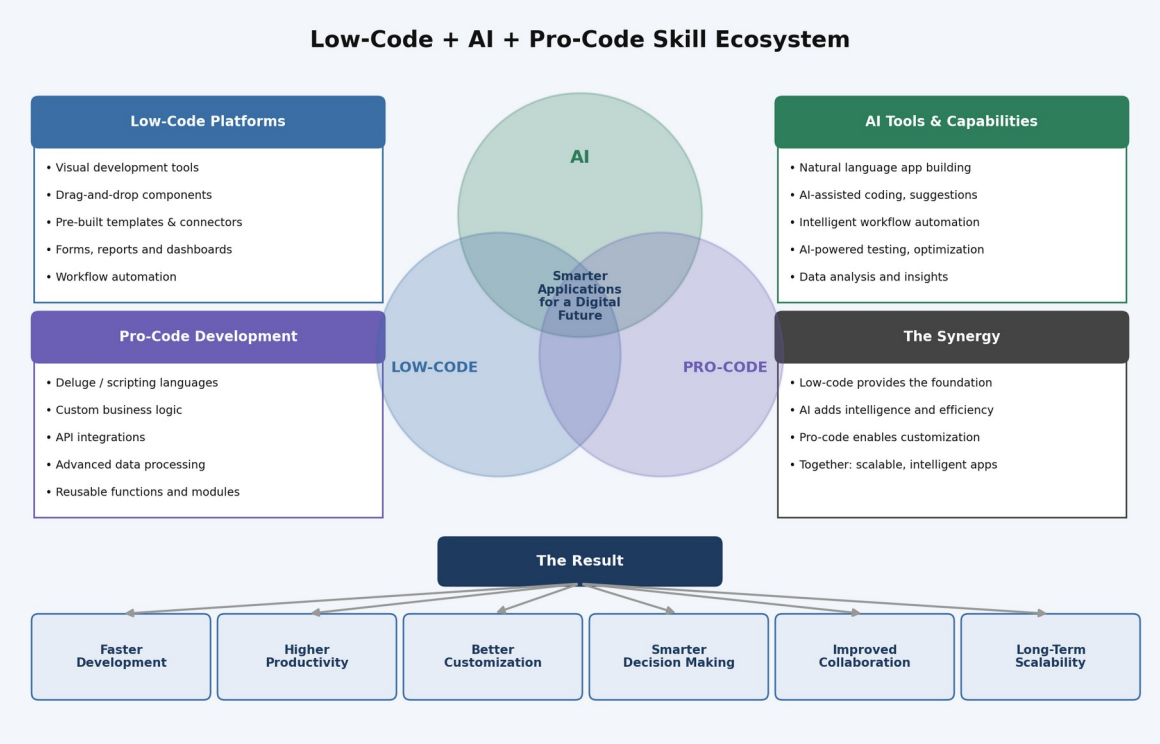



***Figure 3: Integration of low-code platforms, AI capabilities and pro-code skills.***

## Future Developer Skill Set

The future developer is expected to have a combination of the following:

**Low-Code Platforms + Programming Fundamentals + APIs & Integration + Databases + AI Tools + Business Understanding**

| Component | Role |
|---|---|
| Low-Code Platforms | Building applications quickly using visual tools, components and workflows. |

| Component | Role |
|---|---|
| Programming Fundamentals | Writing custom logic, understanding how applications work and troubleshooting problems. |
| APIs & Integration | Connecting applications with other systems and services. |
| Databases | Organising and managing application data. |
| AI Tools | Assisting with development, automation, testing and data analysis. |
| Business Understanding | Ensuring that applications solve real business needs. |

Low-code platforms can reduce repetitive development effort, while programming knowledge and technical understanding remain valuable for customisation, troubleshooting, integration and advanced solutions.

## Advantages of Low-Code Development

- **Faster development:** visual tools and ready components shorten development time.
- **Reduced repetitive coding:** the platform handles much of the routine programming.
- **Faster prototyping:** working versions of an application can be produced early and refined.
- **Workflow automation:** repetitive business processes can be automated.
- **Easier application maintenance:** visual configuration can make applications easier to understand and modify.
- **Improved collaboration:** technical and business teams can work together more easily on the same application.
- **Wider access to application development:** people with limited programming experience can contribute.
- **Faster response to business requirements:** applications can be changed quickly as needs change.

## Challenges of Low-Code Development

- **Platform limitations:** a platform may not support every requirement.
- **Vendor dependency:** the application depends on the capabilities, pricing and restrictions of the chosen platform.
- **Complex customization requirements:** highly specialised needs may exceed standard components.
- **Integration challenges:** connecting with some systems, especially legacy systems, can be difficult.
- **Scalability considerations:** very large data volumes and user numbers need careful evaluation.
- **Security and governance:** access control, compliance and management of many applications need attention.
- **Need for technical knowledge:** advanced applications still require technical understanding.
- **Requirement for pro-code extensions:** complex scenarios may need custom code in addition to low-code.

These challenges do not make low-code unsuitable. They indicate that the approach should be chosen according to the requirements of the application.

## Career Opportunities

Low-code professionals can work in organisations and areas such as the following:

- Enterprise application development
- Business process automation
- Digital transformation
- IT consulting
- Workflow automation
- System integration
- Cloud application development
- Internal business tools
- Data-driven applications

Because low-code platforms are used to automate and digitise business processes, career opportunities can exist across different industries.

## Future Career Outlook

Low-code development is likely to evolve rather than simply replace conventional software development. Several areas are expected to grow in importance: AI-assisted development, hybrid low-code and pro-code development, integration skills, cloud technologies, data management, business understanding and solution architecture.

As platforms and AI tools take over more repetitive implementation, future developers may spend less time on routine work and more time on problem solving, architecture, customisation, integration and business outcomes.

## Conclusion

Low-code development provides an accessible and efficient approach to application development, and it is creating a growing range of career opportunities. Visual tools, reusable components and automation allow applications to be delivered faster and allow both professional developers and business users to contribute. Career progression can begin with low-code application development and advance towards automation, integration, consulting, solution development and architecture. At each stage, technical depth and responsibility increase, and skills in databases, APIs, programming, automation and business understanding become more important.

The future of low-code development will be strongly influenced by AI, automation, APIs, cloud technologies and collaboration between low-code and pro-code approaches. Developer roles are expected to evolve rather than disappear. A strong future-ready developer should therefore combine low-code platform knowledge with programming fundamentals, databases, APIs, AI tools and business understanding.

24A0182